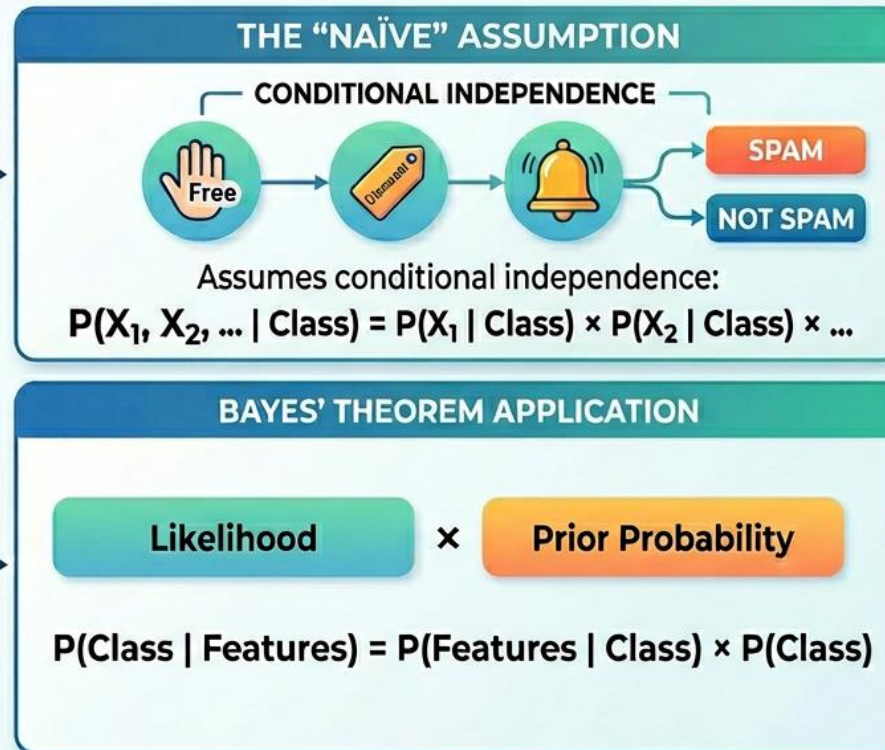


Naïve Bayes Classifier

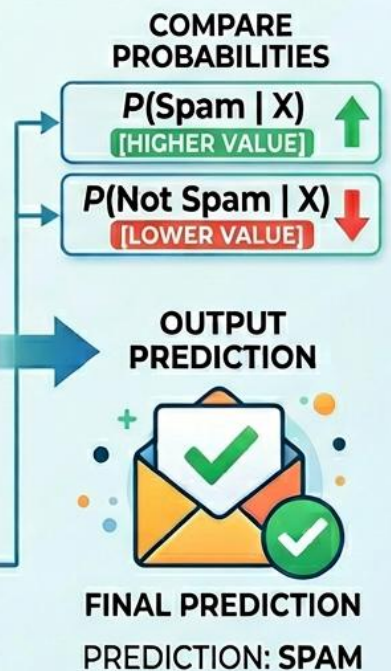
1. INPUT INSTANCE (X)



2. CORE PROCESS



3. OUTPUT PREDICTION



Naïve Bayes Classifier 🤖

- A supervised **probabilistic** classifier based on Bayes Theorem
- Predicts the **most likely** class for a new instance from its features
 - Example: {spherical, yellow, weight < 60 g} → tennis ball 🎾
- It computes the probability of each class by combining:
 - the **prior probability** of the class, and
 - the **conditional probability** of the feature values given that class
- Assumes features are conditionally independent given the class ("naïve" assumption)
- Simple, fast, and effective for binary and multi-class classification ⚡

Naïve Bayes Classifier - Applications

- **Text classification:** spam filtering, sentiment analysis, topic labeling  
- **Medical diagnosis:** disease prediction and risk assessment from symptoms, test results, and patient data 
- **Recommendation systems:** personalized products or content 
- **Fast baseline classifier:** simple, scalable, and efficient on high-dimensional data 

Example of Bayes' Theorem 🩺 🤒

- **Goal:** Find the probability that a patient has Flu, given that the patient has a Headache: Find **P(Flu | Headache)**
- **Given:**
 - $P(\text{Flu}) = 1/40 \rightarrow$ Flu is relatively rare 🤒
 - $P(\text{Headache}) = 1/10 \rightarrow$ Headache is more common 🤒
 - $P(\text{Headache} | \text{Flu}) = 1/2 \rightarrow$ 50% of flu patients have headache 🩺
- **Bayes' theorem:**

$$\begin{aligned} P(\text{Flu} | \text{Headache}) &= \frac{P(\text{Headache} | \text{Flu}) \times P(\text{Flu})}{P(\text{Headache})} \\ &= \frac{\frac{1}{2} \times \frac{1}{40}}{\frac{1}{10}} = 12.5\% \quad \checkmark \end{aligned}$$



Thomas Bayes

Conclusion:

Even if a patient has a **Headache**, the probability of **Flu** is only **12.5%**. ✓

- **Goal:** Estimate the **probabilities** needed to classify a new patient 📊

- **Class labels:**

F = patient has **Flu** 🤧

N = patient has **No Flu** 😊

- **Features used:** Fever 🌡️ & Cough 🤧

- **Training set: 6 patients** 👥

- **Class Prior probabilities:**

$$P(F) = 3/6 = 0.5$$

$$P(N) = 3/6 = 0.5$$

- **Conditional feature probabilities given Flu (F):**

🌡️ $P(\text{Fever}=\text{Yes} \mid F) = 2/3$

🤧 $P(\text{Cough}=\text{Yes} \mid F) = 2/3$

😊 $P(\text{Fever}=\text{No} \mid F) = 1/3$

😊 $P(\text{Cough}=\text{No} \mid F) = 1/3$

- **Conditional feature probabilities given No Flu (N):**

🌡️ $P(\text{Fever}=\text{Yes} \mid N) = 1/3$

🤧 $P(\text{Cough}=\text{Yes} \mid N) = 1/3$

😊 $P(\text{Fever}=\text{No} \mid N) = 2/3$

😊 $P(\text{Cough}=\text{No} \mid N) = 2/3$

Naïve Bayes Example: Flu Prediction 🤧🩺

Fever 🌡️	Cough 🤧	Class 🏷️
Yes	Yes	Flu 🤧
Yes	No	Flu 🤧
No	Yes	Flu 🤧
No	Yes	No Flu ✅
No	No	No Flu ✅
Yes	No	No Flu ✅

These probabilities are the **statistics learned** from the training data and will be used to **classify a new patient** 🎯

Classify new Patient 🤒 🩺

- Classify a new patient **A** using the Naïve Bayes classifier

Fever 🌡️	Cough 🤧	Class 🏷️
Yes	Yes	?

- Calculate **$P(F | A)$** and **$P(N | A)$** aka **Posterior probability**

Combine likelihoods with prior probabilities:

$$P(F | A) \approx P(A | F) \times P(F) / P(A)$$

$P(A | F)$ aka **Likelihood**, is computed by multiplying the feature probabilities given **F**, under the Naïve Bayes **conditional independence assumption**

$$\approx P(\text{Fever=Yes} | F) \times P(\text{Cough=Yes} | F) \times P(F)$$

$$\approx \frac{4}{9} \times \frac{1}{2} = \frac{2}{9}$$

$$P(N | A) \approx P(A | N) \times P(N) / P(A)$$

$$\approx \frac{1}{9} \times \frac{1}{2} = \frac{1}{18}$$

$P(A)$ is the same for every class, so we **ignore it** and compare only the numerators

Since **$P(F | A) > P(N | A)$** => Therefore, assign the patient A class **Flu (F)** 🤒

Naïve Bayes predicts the **class** with the **highest probability**

Naïve Bayes Classifier

- Let $\mathbf{X} = (x_1, x_2, \dots, x_m)$ be the feature vector of an instance
- Let Y be the class label, where $Y \in \{y_1, y_2, \dots, y_k\}$
- **Goal:** For each class $c \in \{y_1, y_2, \dots, y_k\}$ compute the **posterior probability** $P(Y = c | \mathbf{X})$ given the features of $\mathbf{X}$ using **Bayes' theorem**:

$$P(Y = c | \mathbf{X}) = \frac{P(\mathbf{X} | Y = c) P(Y = c)}{P(\mathbf{X})}$$

with:

- $P(Y = c | \mathbf{X})$: **posterior** probability for class c
- $P(\mathbf{X} | Y = c)$: **likelihood** of $\mathbf{X}$ given class c
- $P(Y = c)$: **prior** probability for class c
- $P(\mathbf{X})$: **evidence** (same for all classes so it can be ignored)

Apply the Naïve Assumption and Compare Class Probabilities

- For a given input X , $P(X)$ is the same for every class. So, we **ignore it** and **compare only the numerators**
- Naïve Bayes assumes the features are **conditionally independent** given class c , thus the likelihood of the whole feature vector X is obtained by **multiplying the conditional probabilities of the individual features $x_1, x_2, \dots, x_m$ given class c**

$$P(X | Y = c) = P(x_1 | Y = c) P(x_2 | Y = c) \dots P(x_m | Y = c)$$

$$= \prod_{j=1}^m P(x_j | Y = c)$$

where $\mathbf{x}_j$ is the **j -th** feature value of the input X

Therefore, for each class c , we compute $P(Y = c | X) \approx P(Y = c) \prod_{j=1}^m P(x_j | Y = c)$

- **Predicted class** is the one with the largest probability:

$$\hat{y} = \arg \max_c P(Y = c) \prod_{j=1}^m P(x_j | Y = c)$$

Naïve Bayes Classifier for the Credit Example

- Class labels: {good, bad}
 - $P(\text{Class} = \text{good}) = 19/27 = 0.7$
 - $P(\text{Class} = \text{bad}) = 8/27 = 0.3$
- Conditional Probabilities
 - $P(\text{Housing} = \text{own} \mid \text{bad}) = 4/8 = 0.5$
 - $P(\text{Housing} = \text{own} \mid \text{good}) = 13/19 = 0.68$
 - $P(\text{Housing} = \text{rent} \mid \text{bad}) = 4/8 = 0.5$
 - $P(\text{Housing} = \text{rent} \mid \text{good}) = 6/19 = 0.316$
 - ... and so on

Job	Housing	Savings	Credit_class
self_employed	own	low	good
skilled	own	low	good
skilled	own	high	good
skilled	own	high	good
skilled	own	high	good
skilled	rent	low	good
skilled	own	low	good
skilled	rent	low	good
skilled	own	high	good
skilled	own	high	good
skilled	rent	low	good
skilled	rent	low	good
skilled	own	high	good
skilled	rent	low	good
skilled	own	high	good
skilled	own	high	good
skilled	own	low	bad
self_employed	rent	low	bad
self_employed	own	low	bad
self_employed	own	high	bad
self_employed	rent	low	bad
self_employed	rent	low	bad
self_employed	rent	low	bad
self_employed	own	high	bad

Naïve Bayes Classifier for a Particular Applicant

- Given applicant attributes of

A = {Housing = own, Job = self-employed, savings = low}

- We need to compare $P(\text{bad}|A)$ against $P(\text{good}|A)$, and we also need to calculate $P(A|\text{bad})$ and $P(A|\text{good})$

$$P(\text{good}|A) \approx P(A|\text{good}) \times P(\text{good})$$

$$P(\text{bad}|A) \approx P(A|\text{bad}) \times P(\text{bad})$$

$$P(\text{good}|A) \approx P(A|\text{good}) \times 0.7$$

$$P(\text{bad}|A) \approx P(A|\text{bad}) \times 0.3$$

a_j	C_i	$P(a_j C_i)$
Housing = own	good	$13/19 = 0.68$
Housing = own	bad	$4/8 = 0.5$
Job = self emp	good	$1/19 = 0.05$
Job = self emp	bad	$6/8 = 0.75$
Savings = low	good	$9/19 = 0.73$
Savings = low	bad	$7/8 = 0.875$

$$\begin{aligned} P(A | \text{good}) &= P(\text{Housing} = \text{own} | \text{good}) \times P(\text{Job} = \text{self-employed} | \text{good}) \times P(\text{Savings} = \text{low} | \text{good}) \\ &= 0.68 \times 0.05 \times 0.73 = 0.2482 \end{aligned}$$

$$\begin{aligned} P(A | \text{bad}) &= P(\text{Housing} = \text{own} | \text{bad}) \times P(\text{Job} = \text{self-employed} | \text{bad}) \times P(\text{Savings} = \text{low} | \text{bad}) \\ &= 0.5 \times 0.75 \times 0.875 = 0.3281 \end{aligned}$$

$$P(\text{good}|A) \approx 0.2482 \times 0.7 = 0.0174$$

$$P(\text{bad}|A) \approx 0.3281 \times 0.3 = 0.0984$$

Since $P(\text{bad}|A) > P(\text{good}|A)$, assign the applicant the label **"bad"** credit

Handling Zero Probabilities with Laplace Smoothing

- Zero probabilities due to unobserved feature/class pairs
 - If one conditional probability is 0, then the whole class score becomes 0 because the conditional probabilities are multiplied together
- Can be corrected using **Laplace smoothing**
 - Adds 1 to each possible feature value so that no probability becomes zero
 - Since Body Ache is a binary feature (Yes / No), the denominator is increased by 2

Fever 	Cough 	Body Ache 	Class 
Yes	Yes	Yes	Flu 
Yes	No	Yes	Flu 
No	Yes	No	Flu 
No	Yes	No	No Flu 
No	No	No	No Flu 
Yes	No	No	No Flu 

	Without smoothing	Laplace smoothing
$P(\text{BodyAche}=\text{Yes} \mid \text{F})$	$2/3$	$(2 + 1) / (3 + 2) = 3/5$
$P(\text{BodyAche}=\text{Yes} \mid \text{N})$	0/3	$(0+1) / (3+2) = 1/5$

Key idea:

- Without smoothing → one zero cancels the class score
- **With smoothing** → probabilities stay small, but never zero => makes Naïve Bayes more robust when some feature/class combinations are missing from the training data

New patient $X = \{\text{Fever}=\text{Yes}, \text{Cough}=\text{Yes}, \text{BodyAche}=\text{Yes}\} \Rightarrow \text{Class?}$

- **Without smoothing, the class No Flu becomes impossible, even if other evidence may support it**
- **After smoothing:**

$$\begin{aligned}\text{Score}(N) &= P(N) P(\text{Fever}=\text{Yes} \mid N) P(\text{Cough}=\text{Yes} \mid N) P(\text{BodyAche}=\text{Yes} \mid N) \\ &= 1/2 \times 1/3 \times 1/3 \times 1/5 = 1/90\end{aligned}$$

Laplace Smoothing of Conditional Probabilities

- **Purpose:** avoid zero probabilities when a feature value is rare or was not observed in the training data

With Laplace smoothing

$$P(x_i = v \mid Y = c) = \frac{N_{vic} + 1}{N_c + k_i}$$

Prevents any conditional probability from being zero
Makes Naïve Bayes **more robust** when some feature/class combinations are missing from the training data

Where

- N_{vic} : count of training examples of class c where feature x_i takes value v
- N_c : count of training examples belonging to class c
- k_i : number of **possible values (categories)** of feature x_i

Example

If **Body Ache** is binary (**Yes / No**), then $k_i = 2$:

$$P(\text{BodyAche=Yes} \mid \text{No Flu}) = \frac{0 + 1}{3 + 2} = \frac{1}{5}$$

So the probability becomes **small, but not zero**

Naïve Bayes Implementation: Avoid Numerical Underflow

- Naïve Bayes multiplies many small probabilities
 - With many features, the product can become so small that it is rounded to 0
 - This is called **numerical underflow**
- **Solution:**
 - Use log-probabilities instead of raw probabilities
 - Instead of multiplying probabilities, we add their logarithms
- For each class c , compute the **log-score**:

$$\log P(Y = c \mid X) = \log P(Y = c) + \sum_{j=1}^m \log P(x_j \mid Y = c) - \log P(X)$$

Python

```
log_score_flu = math.log(prior_flu)
                + sum(math.log(p) for p in p_feature_given_flu)
```

Key idea

- **Product of tiny probabilities** → unstable ❌
- **Sum of log-probabilities** → stable ✅

Naïve Bayes Classifier — Summary

Strengths

- Simple, fast, and easy to implement
- Scales well to high-dimensional data, especially in text classification
- Works well as a **strong baseline model** for many classification tasks

Limitations

- The conditional independence assumption is often unrealistic
 - It may perform less well when features are strongly correlated or classes are highly imbalanced
- Continuous features usually need discretization

Takeaway

- Naïve Bayes is a fast, simple, and effective classifier, especially as a baseline model